

Module 19: Virtual Threads

A virtual thread is a thread the JVM schedules rather than the operating system. Same `Thread` API, same code, same debugger. What changed is the price.

A platform thread reserves about a megabyte of stack and an OS scheduling slot. A virtual thread is a heap object that grows as needed. That single change makes blocking cheap again, and this module is mostly about what follows from it.

A warning about other sources: most material written about virtual thread pinning is out of date. This module measures the current behaviour rather than repeating it.

What You'll Learn

- Mounting, unmounting, and what a carrier thread is
- The benchmark that makes the case, measured on your own machine
- What pins today, which is far less than it was
- `ScopedValue`, and why `ThreadLocal` does not scale to a million threads
- Four ways people misuse them, three from old habits

Coming From Another Language

	Go	Python	Java
Lightweight concurrency	goroutine	<code>asyncio</code> task	virtual thread
Blocking call	fine, scheduler handles it	blocks the loop	fine, thread unmounts
Colour problem	none	<code>async</code> / <code>await</code> splits your API	none
Cost each	~2KB	small	small, grows as needed

Virtual threads put Java where Go has been since the start: you write ordinary blocking, sequential code, and the runtime makes it scale. No `async` keyword splits the library in two.

The Lesson

Mounting and carriers

```
Thread.ofVirtual().name("worker").start(() -> ...);
```

Printing the thread shows something like `VirtualThread[#33,worker]/runnable@ForkJoinPool-1-worker-1`. That `ForkJoinPool-1-worker-1` is the **carrier**: a real platform thread the virtual thread is

currently mounted on.

When the virtual thread blocks, it unmounts and the carrier picks up another one. That is the whole mechanism. Blocking stopped costing a thread.

The executor you will actually use creates **one thread per task**, not a pool:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ... }
```

There is nothing to size, because the threads are not the scarce resource. From `VirtualThreads.java`:

```
100000 virtual threads, each sleeping 10ms: 216ms
```

The benchmark

From `Benchmark.java`, 5,000 tasks each blocking for 100ms on an 8-core machine:

virtual thread per task	127ms
platform pool of 200	2716ms
platform pool of 50	10750ms

The arithmetic is simple. A pool of 200 can have at most 200 tasks blocked at once, so 5,000 tasks take 25 batches of 100ms. A pool of 50 takes 100 batches. Virtual threads have no such limit, so all 5,000 block at once.

The choice this removes. Sizing a platform pool for IO was always a bad trade: too few threads and requests queue behind blocked ones, too many and you pay a megabyte of stack each while the OS scheduler thrashes.

The usual escape was asynchronous code, which works and costs you readable stack traces, step debugging, and ordinary `try / catch`. Virtual threads give the async scalability back to plain blocking code.

Where they do not help: CPU-bound work. A task that never blocks never unmounts, and you cannot run more work in parallel than you have cores. A fixed platform pool is still correct there.

Pinning, today

Pinning is when a virtual thread cannot unmount, so its carrier stays blocked and a platform thread is wasted.

In Java 21 to 23, blocking inside a `synchronized` block pinned the carrier, and every article from that era tells you to replace `synchronized` with `ReentrantLock`.

JEP 491, delivered in JDK 24, removed that. From `PinningToday.java`, 2,000 tasks each holding their own lock while sleeping 100ms:

```
cores, and therefore carriers: 8
elapsed: 131ms
```

If synchronized still pinned, only 8 tasks could sleep at once, so this would take about 25000ms.

What still pins:

1. **A native method or foreign function call.** The JVM cannot unmount a stack it does not control.
2. **A class initialiser that blocks.** Rare, and usually a design problem.

Both are far narrower than the old rule.

Pinning and contention are different things. A lock is still a lock: if a thousand virtual threads contend for one monitor, 999 wait. Pinning wastes a *carrier*; contention wastes *time*. JEP 491 fixed the first and could not touch the second, because serialising is what a lock is for.

Check your own code with `-Djdk.tracePinnedThreads=full` rather than trusting any article, including this one.

ScopedValue

Finalised in JDK 25. It replaces `ThreadLocal` for the common case.

The problem: a `ThreadLocal` is a mutable, unbounded, per-thread entry that lives until someone remembers to `remove()` it. With a few hundred pooled threads that was survivable. With a million virtual threads it is not.

```
private static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, "ada").run(() -> {
    handleRequest();           // CURRENT_USER.get() works at any depth
});
// binding is gone here, including if the body threw
```

There is no `remove()` to forget. The binding lasts exactly as long as `run()`.

It is immutable, which is the point. A `ThreadLocal` can be set by anything that can reach it, at any depth, and the change persists for the thread's life. A `ScopedValue` can only be shadowed by a nested scope, which is visible in the code.

One limitation worth knowing:

```
child thread sees it bound? false
```

A plain child thread does **not** inherit the binding, deliberately: the parent's scope could end while the child still runs. Inheritance requires structured concurrency, which is JEP 533 and still in preview on this

release. Module 22 covers it.

Use	For
<code>ScopedValue</code>	read-only value for the duration of a call: current user, request id
<code>ThreadLocal</code>	genuinely mutable per-thread state: a reusable buffer

Four misuses

From `CommonMisuses.java`:

- 1. Pooling them.** `newFixedThreadPool(200, Thread.ofVirtual().factory())` caps you at 200 and gains nothing. Pools reuse an expensive resource; these are not expensive.
- 2. Forgetting the pool was also your rate limit.** A platform pool did two jobs: providing threads and bounding concurrency. Virtual threads replace only the first. If a pool of 10 was protecting your database, removing it removes the protection, and you find out at 3am. Use a `Semaphore`, which states the constraint instead of implying it.
- 3. CPU-bound work.** Ten thousand virtual threads doing arithmetic will not beat your core count; they add scheduling overhead.
- 4. `ThreadLocal` at scale.** One entry per thread, and now there are a million.

Virtual threads make blocking cheap. They do not make sharing safe, and they are not a CPU multiplier. Everything in Module 18 about races, visibility and locks applies word for word.

Run It

```
java modules/19-virtual-threads/examples/VirtualThreads.java
java modules/19-virtual-threads/examples/Benchmark.java
java modules/19-virtual-threads/examples/PinningToday.java
java modules/19-virtual-threads/examples/ScopedValues.java
java modules/19-virtual-threads/examples/CommonMisuses.java

./scripts/verify-examples.sh modules/19-virtual-threads
```

The benchmark takes about fifteen seconds, most of it deliberately spent waiting in the platform-pool cases.

Common Mistakes

Pooling virtual threads. One per task; there is nothing to reuse.

Losing an unintended rate limit. Bound concurrency explicitly with a `Semaphore`.

Using them for CPU-bound work. Cores are still the limit.

Keeping `ThreadLocal` at scale. One entry per thread, and there are now millions.

Following advice to avoid `synchronized`. That was true before JDK 24 and is not now. Measure rather than assume.

Assuming they fix concurrency bugs. They fix thread *cost*. Races and visibility are untouched.

Expecting a child thread to inherit a `ScopedValue`. It does not, without structured concurrency.

Key Takeaways

- **A virtual thread unmounts when it blocks**, freeing its carrier. That is the whole idea.
- **One thread per task**, not a pool.
- **JEP 491 removed `synchronized` pinning in JDK 24.** Native calls still pin.
- **Pinning wastes a carrier; contention wastes time.** Different problems.
- **`ScopedValue` is immutable and scope-bounded**, with nothing to clean up.
- **They make blocking cheap, not sharing safe.** Module 18 still applies in full.

Homework

[homework/README.md](#)

Measure the difference yourself, then find the rate limit you accidentally removed. Reference solution in [solutions/19-virtual-threads/](#).